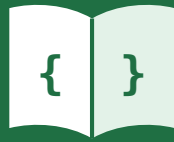

Modül 15 — Gözlemlenebilirlik: Metrikler, Trace'ler, Loglar

Ders Notları

Spring Boot Edu



2026-09-24

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Üç Ayak	2
2.2 Micrometer ve Observation API	2
2.3 Trace'ler ve Span'ler	3
2.4 OTLP ve Grafana LGTM	3
3. Adım Adım Örnekler	3
3.1 Actuator ve Health	4
3.2 Micrometer ile İş Metrikleri	5
3.3 Micrometer'dan Prometheus'a	6
3.4 @Observed	6
3.5 Servisler Arası Trace'ler ve Log Korelasyonu	7
3.6 Grafana	8
3.7 Gözlemlenebilirliği Test Etmek	8
4. Sık Yapılan Hatalar ve En İyi Pratikler	9
5. Özet	10
6. İleri Okuma	10

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Gözlemlenebilirliğin üç ayağını açıklamak: metrikler, trace'ler ve loglar
- Actuator'ı güvenle kullanmak: özel indicator'lı health, probe'lar, metrikler ve gereken-den fazlası değil
- Micrometer ile iş metrikleri kaydetmek: counter, timer, gauge
- Bir metodu @Observed ile gözlemlemek ve tek seferde bir timer ile bir span elde etmek
- OpenTelemetry tracing ile bir isteği servisler arasında izlemek
- Trace id taşıyan yapılandırılmış JSON loglar yazmak
- Hepsini Grafana LGTM'ye göndermek ve bir dashboard'da okumak

Ön koşullar: Modül 03 (REST) · **Tahmini süre:** 5 saat · Grafana için **Docker gerekir**

2. Kavramlar

2.1 Üç Ayak

Sinyal	Yanıtladığı	Örnek	Saklandığı yer (LGTM)
Metrikler	Ne kadar? Ne hızlı? Ne sıklıkla?	dakikadaki sipariş, p95 gecikme	Prometheus/Mimir
Trace'ler	Bir isteğin süresi nereye gitti?	HTTP → fiyatlandırma → katalog	Tempo
Loglar	Tam olarak ne oldu?	"order placed for 9780134685991"	Loki

Sinyaller **birbirine bağlı** olduklarında en yararlıdır: Bir metriktaki gecikme sıçraması ya-vaş bir trace'e, trace id ise o isteğin log satırlarına götürür.

2.2 Micrometer ve Observation API

Micrometer, metrikler için SLF4J'nin loglama için olduğu şeydir: tek bir API, birçok backend. **Observation API**'si bir adım daha ileri gider: Bir işlemi bir kez tanımlarsınız (Observation veya @Observed) ve handler'lar onu bir timer'a **ve** bir span'e dönüştürür. Spring MVC, RestClient, JDBC ve daha birçoğu hazır olarak gözlemlenir.

2.3 Trace'ler ve Span'ler

Bir **trace**, bir isteğin tüm yolculuğudur. **Span**'lerden oluşur: her adım için bir tane, her birinin bir başlangıcı, bir süresi ve bir ebeveyni vardır. Trace id, W3C traceparent HTTP header'ı ile diğer servislere taşınır. Böylece bir trace birçok uygulamayı kapsayabilir.

2.4 OTLP ve Grafana LGTM

OTLP (OpenTelemetry Protocol), metrikleri, trace'leri ve logları göndermenin üreticiden bağımsız yoludur. Spring Boot 4'ün spring-boot-starter-opentelemetry'si metrikleri ve trace'leri OTLP ile dışa aktarır. compose.yaml'daki grafana/otel-lgtm image'ı Loki, Grafana, Tempo ve Mimir/Prometheus ile birlikte bir OpenTelemetry Collector içerir. Geliştirme için idealdir, ancak bir üretim kurulumu değildir.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. Grafana LGTM kök dizindeki compose.yaml dosyasından başlar ve Boot metrikleri ile trace'leri ona otomatik gönderir:

```
./mvnw -pl modules/15-observability/lesson spring-boot:run
```

Sonra birkaç sipariş verin (bkz. requests.http) ve Grafana'yı http://localhost:3000 adresinde açın.

Yapılandırma:

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics          # never "": heapdump, env, ... leak
    ↪ internals
  endpoint:
    health:
      show-details: always                      # fine locally; in production:
    ↪ when-authorized
      probes:
        enabled: true                          # /actuator/health/liveness and /readiness
  info:
    java:
      enabled: true
  observations:
    annotations:
```

```

    enabled: true                                # @Observed
    tracing:
      sampling:
        probability: 1.0                        # every request is traced (production:
↪ e.g. 0.1)
      metrics:
        tags:
          application: ${spring.application.name} # a common tag on every metric
    otlp:
      metrics:
        export:
          step: 10s                             # push metrics every 10 s (default: 1
↪ minute)
    logging:
      structured:
        format:
          console: ecs                          # one JSON object per line (Elastic
↪ Common Schema)

```

3.1 Actuator ve Health

Actuator /actuator altına operasyonel endpoint'ler ekler. Burada yalnızca health, info ve metrics açılır. env veya heapdump gibi endpoint'ler yapılandırma değerlerini ve bellek içeriğini açığa çıkarırdı.

Özel bir indicator, health durumuna bir bileşen ekler:

```

@Component
class WarehouseHealthIndicator implements HealthIndicator {

    private final Warehouse warehouse;

    WarehouseHealthIndicator(Warehouse warehouse) {
        this.warehouse = warehouse;
    }

    @Override
    public Health health() {
        return warehouse.reachable()
            ? Health.up().withDetail("location", "Istanbul").build()
            : Health.down().withDetail("reason", "no connection to the warehouse
↪ system").build();
    }
}

```

- Bileşen adı bean adından gelir: warehouseHealthIndicator → warehouse.

- Herhangi bir bileşen DOWN ise tüm durum DOWN olur ve endpoint **503** ile yanıt verir. Böylece bir load balancer bu örneğe trafik göndermeyi bırakır.
- /actuator/health/liveness ve /actuator/health/readiness, Kubernetes probe'larının kullandığı gruplardır (modül 22).

! Uyarı

show-details: always yerelde kullanışlıdır. Üretimde when-authorized kullanın: Health ayrıntıları host adlarını ve sürümleri açığa çıkarabilir.

3.2 Micrometer ile İş Metrikleri

```
@Component
public class OrderMetrics {

    private final MeterRegistry registry;
    private final Timer processing;
    private final AtomicInteger inProgress = new AtomicInteger();

    public OrderMetrics(MeterRegistry registry) {
        this.registry = registry;
        this.processing = Timer.builder("bookstore.order.processing")
            .description("Time to place an order")
            .publishPercentiles(0.5, 0.95) // median and p95
            .register(registry);
        Gauge.builder("bookstore.orders.in.progress", inProgress, AtomicInteger::get)
        ↪ // current value, read on demand
            .register(registry);
    }

    public <T> T recordOrder(String channel, Supplier<T> placeOrder) {
        inProgress.incrementAndGet();
        try {
            T result = processing.record(placeOrder);
            Counter.builder("bookstore.orders.placed")
                .tag("channel", channel) // few values
                ↪ only: web, app
                .register(registry) // returns the
                ↪ existing counter
                .increment();
            return result;
        } finally {
            inProgress.decrementAndGet();
        }
    }
}
```

Metre	Ölçtüğü	Burada
Counter	bir şeyin kaç kez olduğu (yalnızca artar)	verilen siparişler, kanal başına
Timer	ne kadar sürdüğü ve ne sıklıkla olduğu	sipariş işleme, medyan ve p95 ile
Gauge	yayınlandığı anda okunan anlık bir değer	işlemdeki siparişler

Tag'ler bir metreyi serilere böler, ör. channel=web ve channel=app. Her farklı tag değeri yeni bir zaman serisi oluşturur. Bu yüzden tag'lerin **az** sayıda olası değeri olmalıdır. Tag olarak bir müşteri id'si veya ISBN, bir metrik backend'ini çökertebilir (yüksek kardinalite).

```
curl 'localhost:8080/actuator/metrics/bookstore.orders.placed?tag=channel:web'
```

3.3 Micrometer'dan Prometheus'a

OTLP üzerinden Micrometer'ın adları, birimi sonek olarak alan Prometheus adlarına dönüşür. LGTM yığımına ulaşan seriler şunlardır:

Micrometer	Prometheus
bookstore.orders.placed (counter)	bookstore_orders_placed_total
bookstore.order.processing (timer)	bookstore_order_processing_milliseconds_count, ..._sum, ...{quantile="0.95"}
bookstore.orders.in.progress (gauge)	bookstore_orders_in_progress
http.server.requests (Spring MVC'den)	http_server_requests_milliseconds_count, ..._bucket, ...

management.metrics.tags ile verilen ortak application tag'i her seride bulunur.

3.4 @Observed

```
@Service
public class PricingService {
```

```

@Observed(name = "bookstore.pricing",           // metric name
           contextualName = "calculate-price",   // span name
           lowCardinalityKeyValues = {"currency", "TRY"}) // tag on both
public BigDecimal priceOf(String isbn) {
    sleep(50); // pretend to ask a
    ↪ pricing engine
    return isbn.endsWith("1") ? new BigDecimal("89.90") : new BigDecimal("55.00");
}

```

Tek bir annotation, bookstore.pricing timer'ını (currency=TRY tag'iyle) ve bu metottan geçen her trace'te bir calculate-price span'i verir. spring-boot-starter-aspectj ve management.observations.annotations.enabled=true ister.

3.5 Servisler Arası Trace'ler ve Log Korelasyonu

Sipariş controller'ı "katalog servisini" HTTP ile çağırır. RestClient, Boot'un RestClient.Builder'ından kurulduğu için çağrı gözlemlenir ve trace bağlamı birlikte gönderilir:

```

@Component
public class CatalogClient {

    private final RestClient.Builder http;
    private final Environment environment;

    public CatalogClient(RestClient.Builder http, Environment environment) { // the
    ↪ builder, not RestClient.create()
        this.http = http;
        this.environment = environment;
    }

    public String titleOf(String isbn) {
        String port = environment.getProperty("local.server.port", "8080");
        Map<?, ?> book = http.baseUrl("http://localhost:" + port).build()
            .get().uri("/api/catalog/{isbn}", isbn)
            .retrieve().body(Map.class);
        return String.valueOf(Objects.requireNonNull(book).get("title"));
    }
}

```

Bu yüzden bir sipariş tek bir trace üretir:

```

POST /api/orders                (server span)
├─ calculate-price              (@Observed, ≈ 50 ms)

```

```

└ GET /api/catalog/{isbn}      (client span)
  └ GET /api/catalog/{isbn}    ("diğer servisin" server span'i)

```

logging.structured.format.console: ecs ile her log satırı bir JSON nesnesidir. Bir istek sırasında trace ve span id'sini de taşır:

```

{"@timestamp":"2026-09-
  ↪ 24T19:27:17.777Z","log":{"level":"INFO","logger":"...OrderController"},
  "message":"order placed for 9780134685991 via web",
  "traceId":"349b7da28c9e1f1eff65fdb9c95f36644","spanId":"14d5b6755a51c2c5", ...}

```

Controller'ın ve kataloğun log satırları **aynı** traceId'yi taşır. TraceCorrelationTest tam olarak bunu kontrol eder. Loglarınızda Grafana'dan aldığınız bir trace id'yi arayın, o istekte olan her şeyi bulursunuz.

i Not

Boot 4 bir OTLP log exporter'ı da hazırlar, ancak log olayları ona yalnızca bir OpenTelemetry Logback appender'ı üzerinden ulaşır. Bu appender Spring Boot'un BOM'u tarafından yönetilmez ve bu kursta kullanılmaz. Burada loglar konsola JSON olarak gider. Üretimde bir log toplayıcı (ör. OpenTelemetry Collector veya Grafana Alloy) onları Loki'ye taşır.

3.6 Grafana

Grafana'da (<http://localhost:3000>):

- **Explore → Tempo:** 15-observability'nin trace'lerini arayın ve yukarıdaki span'leri bir zaman çizelgesi olarak görmek için birini açın.
- **Explore → Prometheus:** ör. `sum by (channel) (rate(bookstore_orders_placed_total[1m])) * 60`.

Modül beş panelli hazır bir dashboard içerir (kanal başına dakikadaki siparişler, işleme süresi, fiyatlandırma süresi, HTTP istekleri, işlemdeki siparişler). Bir kez içe aktarın:

```

curl -X POST -H 'Content-Type: application/json' \
  -d @modules/15-observability/grafana/bookstore-orders-dashboard.json \
  http://localhost:3000/api/dashboards/db

```

3.7 Gözlemlenebilirliği Test Etmek

- Testlerde Boot observation'ları tutar ama metrikleri ve trace'leri **dışa aktarmaz**. @AutoConfigureTracing, trace id'ler hakkında doğrulamalar için gerçek bir tracer verir.

- `TestObservationRegistry` (`micrometer-observation-test`'ten) `observation`'ları ve ebeveynlerini hiçbir backend olmadan kontrol eder (Ödev 2).
- `GrafanaLgtmIT` tüm zinciri test eder: bir `Testcontainers LGTM` yığını, yeniden açılan `export` ve `Prometheus` ile `Tempo`'ya karşı doğrulamalar:

```
@TestConfiguration(proxyBeanMethods = false)
static class Lgtm {

    static final LgtmStackContainer LGTM = new
        ↪ LgtmStackContainer("grafana/otel-lgtm:0.33.1"); // as compose.yaml

    @Bean
    @ServiceConnection // OTLP endpoints for metrics,
        ↪ traces and logs
    LgtmStackContainer lgtm() {
        return LGTM;
    }
}
```

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Sınırsız değerleri (kullanıcı id'leri, ISBN'ler, id içeren URL'ler) asla metrik tag'i olarak kullanmayın. Her değer yeni bir zaman serisi oluşturur ve metrik backend'inin belleği biter. Bu tür değerleri trace'lere veya loglara koyun.

- **Yapın:** Yalnızca ihtiyacınız olan Actuator endpoint'lerini açın ve onları koruyun (ayrı port, Spring Security).
- **Yapmayın:** Yoğun bir üretim sisteminde trace'lerin %100'ünü örneklemeyin. Küçük bir `sampling.probability` ile başlayın ve gereken yerde artırın.
- **Yapın:** Metrikleri ölçtükleri şeyin adıyla, noktalarla adlandırın (`bookstore.orders.placed`). `Micrometer` adları her backend için dönüştürür.
- **Yapmayın:** Bir mesajı loglayıp bir de logları ayrıştırarak saymayın. Bir metrikle sayın, bir log satırıyla açıklayın.
- **Yapın:** Logları yapılandırılmış (JSON) yapın ve trace id'yi ekleyin. Böylece loglar, trace'ler ve metrikler birbirine bağlanabilir.
- **Yapmayın:** Bir health indicator'ın yavaş veya kararsız sistemleri zaman aşımı olmadan çağırmasına izin vermeyin. Yavaş bir health kontrolü tüm örneği sağlıklı gösterir.

5. Özet

- Metrikler ne kadar, trace'ler nerede, loglar ne olduğunu söyler. Trace id ile bağlandıklarında bir üretim problemini açıklarlar.
- Actuator health (özel indicator'lar ve probe'larla) ve metrikler verir. Olabildiğince azını açın.
- Micrometer counter, timer ve gauge kaydeder. Tag'lerin az sayıda değeri olmalıdır.
- @Observed ve Spring'in hazır observation'ları timer'ları ve span'leri birlikte üretir.
- RestClient.Builder trace'i sonraki servise taşır. ECS JSON logları traceId ve spanId içerir.
- spring-boot-starter-opentelemetry metrikleri ve trace'leri OTLP ile Grafana LGTM'ye gönderir. Prometheus ve Tempo onları gösterir.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — Actuator · Metrics · Tracing · Observability](#)
- [Spring Boot — Structured Logging](#)
- [Micrometer Documentation · Observation API](#)
- [OpenTelemetry — Concepts](#)
- [Grafana docker-otel-lgtm](#)